T E C H N I C A L D E S I G N D O C U M E N T

Documento de Diseño Técnico

Plataforma de Telemedicina · Arquitectura con WSO2

Versión: 1.0 (borrador para revisión)

Fecha: Octubre 2026

Clasificación: Confidencial

Audiencia: Equipo Técnico y de Producto

Stack de identidad e integración: WSO2

Documento de referencia: Análisis y Requisitos v1.0

Índice de Contenidos

Resumen Ejecutivo.....

Resumen Ejecutivo.....1

1. Introducción y Propósito del Documento.....2

2.1 Alcance del documento.....2

2.2 Relación con el documento de análisis y requisitos.....2

2.3 Audiencia.....3

2.4 Convenciones y notación.....3

2. Decisiones Arquitectónicas (ADRs).....3

2.1 Lista de decisiones registradas.....3

2.2 ADR-001: Plataforma WSO2 unificada.....4

2.3 ADR-002: Microservicios Node.js/NestJS.....4

2.4 ADR-003: PostgreSQL + Redis + OpenSearch.....5

2.5 ADR-004: NATS como bus de eventos.....5

2.6 ADR-005: Patrón BFF con tres backends.....6

2.7 ADR-006: Multi-tenant por columna con RLS.....6

2.8 ADR-007: WSO2 MI como orquestador.....7

3. Visión Arquitectónica.....8

3.1 Diagrama de contexto (C4 nivel 1).....8

3.2 Diagrama de contenedores (C4 nivel 2).....13

3.3 Topología general de red.....14

4. WSO2 Identity Server (Proveedor de Identidad).....14

4.1 Rol y responsabilidades.....14

4.2 Modelo de identidad.....15

4.3 Federación de identidades institucionales.....16

4.4 Autenticación multifactor (MFA).....16

4.5 Autorización: roles, scopes y claims....17
4.6 Provisioning con SCIM 2.0....17
4.7 Consentimiento para aplicaciones SMART on FHIR....18
4.8 Cumplimiento regulatorio de la identidad....18
5. WSO2 API Manager (Gateway y Portal)....18
5.1 Componentes del producto....18
5.2 Catálogo de APIs publicadas....19
5.3 Políticas de seguridad y throttling....19
5.4 Versionado y ciclo de vida de APIs....20
5.5 Portal de developers....20
6. WSO2 Micro Integrator (Orquestación por Servicios)....20
6.1 Rol del orquestador....20
6.2 Topología de integración por servicios....21
6.3 Patrones de integración aplicados....21
6.4 Secuencias de mediación principales....22
6.5 Conectores externos....22
6.6 Manejo de errores, reintentos y compensación....23
6.7 Trazabilidad de los flujos orquestados....23
7. Catálogo de Microservicios de Dominio....23
8.1 Servicio Identidad (servicio-identidad)....23
8.2 Servicio Pacientes (servicio-pacientes)....24
8.3 Servicio Profesionales (servicio-profesionales)....25
8.4 Servicio Agenda y Citas (servicio-agenda)....26
8.5 Servicio Videoconsulta (servicio-video)....26
8.6 Servicio Expediente Clínico (servicio-expediente)....27
8.7 Servicio Recetas (servicio-recetas)....28
8.8 Servicio Pagos y Facturación (servicio-pagos)....29
8.9 Servicio Mensajería (servicio-mensajería)....29

8.10 Servicio Notificaciones (servicio-notificaciones)....30
8.11 Servicio Integraciones EHR (servicio-integraciones)....31
8.12 Servicio Auditoría (servicio-auditoria)....32
8. Modelo de Datos....32
8.1 Bases de datos por servicio....33
8.2 Entidades principales por servicio....33
8.3 Mapeo a FHIR R4....33
9. Comunicación y Eventos....34
9.1 Comunicación sincrónica....34
9.2 Comunicación asíncrona con NATS JetStream....34
9.3 Formato CloudEvents....35
9.4 Patrón outbox para publicación transaccional....35
9.5 Patrón saga coordinada por WSO2 MI....35
9.6 Idempotencia....35
9.7 Catálogo de eventos del bus NATS....36
10. Diagramas de Secuencia de Flujos Críticos....36
10.1 SEQ-01: Reserva de cita con autorización diferida....36
10.2 SEQ-04: Emisión de receta electrónica firmada....39
10.3 SEQ-05: Captura de pago con CFDI....42
10.4 SEQ-07: Exportación FHIR del expediente....44
11. Seguridad Aplicada....47
11.1 Defensa en profundidad....47
11.2 Gestión de secretos....48
11.3 Cifrado y gestión de claves....48
11.4 Auditoría de identidad....48
11.5 Cumplimiento HIPAA, RGPD, NOM-024 e ISO 27001....48
12. Infraestructura y Despliegue....49
12.1 Topología Kubernetes....49

12.2 Namespaces y NetworkPolicy....49
12.3 Helm charts y GitOps....50
12.4 Topología multi-región....50
12.5 Backup y disaster recovery....50
13. Observabilidad....50
13.1 Logs estructurados....51
13.2 Métricas....51
13.3 Trazas distribuidas....51
13.4 Dashboards y alertas....51
14. Plan de Despliegue por Fases....52
14.1 Fase 0 - Descubrimiento y Diseño (4 semanas)....52
14.2 Fase 1 - MVP (12 semanas)....52
14.3 Fase 2 - Beta Cerrada (8 semanas)....52
14.4 Fase 3 - Piloto Productivo (8 semanas)....53
14.5 Fase 4 - Escalamiento (12 semanas)....53
15. Riesgos Técnicos y Mitigaciones....53
Anexo A. Catálogo de APIs Públicas....54
Anexo B. Glosario y Referencias....54
B.1 Glosario de términos técnicos....54
B.2 Referencias técnicas y normativas....54

Nota: Este índice se genera mediante códigos de campo. Para actualizar los números de página tras editar el documento, haga clic derecho sobre el índice y seleccione "Actualizar campo".

Resumen Ejecutivo

El presente documento de diseño técnico (TDD, por sus siglas en inglés) detalla la arquitectura de la plataforma de telemedicina descrita en el documento de análisis y requisitos v1.0, y formaliza dos decisiones clave solicitadas por el patrocinador: la adopción de WSO2 como proveedor de identidad (Identity Provider) y la orquestación de la integración entre servicios mediante el stack WSO2. La propuesta consiste en emplear la plataforma WSO2 en sus tres capacidades complementarias —WSO2 Identity Server como IdP centralizado, WSO2 API Manager como gateway y portal de APIs, y WSO2 Micro Integrator como orquestador de servicios— de modo que la identidad, la exposición de APIs y la mediación entre servicios vivan en una sola plataforma coherente, gobernable y compatible con los marcos regulatorios NOM-024-SSA3-2012, HIPAA, RGPD e ISO/IEC 27001.

La arquitectura de microservicios se mantiene en Node.js con NestJS, PostgreSQL, Redis y NATS, pero ahora los servicios no se comunican directamente entre sí en los flujos transaccionales complejos: el WSO2 Micro Integrator asume el rol de orquestador, ejecutando secuencias de mediación que agregan llamadas a múltiples servicios, aplican transformaciones, gestionan reintentos y compensaciones bajo el patrón saga, y exponen endpoints cohesivos al API Gateway. Los servicios de dominio conservan su autonomía y ownership sobre su base de datos (database per service), pero delegan al orquestador la composición de procesos de negocio que cruzan límites de servicio. El frontend React/Next.js consume exclusivamente las APIs publicadas en WSO2 API Manager, nunca los servicios internos directamente.

El modelo de identidad se rediseña en torno a WSO2 Identity Server. Pacientes, profesionales y administradores son identidades gestionadas por WSO2 IS con atributos clínicos (cédula, especialidad, institución) almacenados como claims. La autenticación multifactor (TOTP y FIDO2/WebAuthn) es obligatoria para profesionales y administradores; el SSO institucional se resuelve mediante federación SAML 2.0 y OIDC hacia los IdPs de las instituciones cliente; el provisioning entre el IdP y los microservicios se realiza por SCIM 2.0; el consentimiento para compartir datos clínicos con aplicaciones SMART on FHIR se gestiona mediante el framework de consentimiento nativo de WSO2. Los tokens JWT emitidos por WSO2 IS son los únicos aceptados por el API Gateway y por los servicios downstream, eliminando la necesidad de un módulo de autenticación propio.

Las principales implicaciones de esta arquitectura son: (1) reducción del acoplamiento entre servicios y por ende del riesgo de cambios disruptivos; (2) gobernabilidad centralizada de seguridad, throttling y versionado de APIs; (3) trazabilidad de extremo a extremo de los flujos orquestados, indispensable para auditoría clínica y cumplimiento regulatorio; (4) reutilización de conectores WSO2 para integraciones externas (FHIR, CFDI/SAT, Stripe, Mercado Pago, WhatsApp, Twilio) en lugar de implementarlos ad-hoc en cada servicio; y (5) un costo operativo adicional por la operación de la plataforma WSO2 que se compensa con la aceleración del desarrollo y la madurez de la postura de seguridad.

El documento se estructura en 16 secciones más apéndices. Las secciones 3 y 4 establecen las decisiones arquitectónicas registradas (ADRs) y la visión general; las secciones 5, 6 y 7 detallan los tres componentes de WSO2; la sección 8 cataloga los microservicios de dominio; las secciones 9 y 10 cubren modelo de datos y comunicación; la sección 11 presenta diagramas de secuencia de los flujos críticos; las secciones 12 a 14 cubren seguridad, infraestructura y observabilidad; la sección 15 conecta el diseño con el roadmap por fases; y la sección 16 identifica los riesgos técnicos. Los apéndices incluyen catálogos de APIs y eventos. El documento es un borrador para revisión del equipo técnico y de producto, y debe leerse en conjunto con el documento de análisis y requisitos v1.0 previamente entregado.

1. Introducción y Propósito del Documento

2.1 Alcance del documento

Este documento de diseño técnico describe la arquitectura de implementación de la plataforma de telemedicina hasta el nivel de componente, interfaz y secuencia. No reabre las decisiones de requisitos funcionales ni de alcance de negocio, que permanecen vigentes según el documento de análisis y requisitos v1.0. Su objetivo es proveer a los equipos de ingeniería, DevOps, seguridad y producto un plano ejecutable que defina qué componentes se construyen, qué componentes se adoptan de terceros (WSO2, Stripe, PAC), cómo se comunican, cómo se despliegan y cómo se operan.

El alcance incluye: la arquitectura de referencia con WSO2 como capa de identidad, APIs e integración; el catálogo de microservicios de dominio; el modelo de datos conceptual por servicio; los patrones de comunicación sincrónica y asincrónica; las secuencias de mediación de los flujos críticos (reserva de cita, videoconsulta, receta electrónica, pago con CFDI, exportación FHIR); la estrategia de despliegue en Kubernetes multi-región; la observabilidad; y el mapeo del diseño a las fases del roadmap.

2.2 Relación con el documento de análisis y requisitos

El documento de requisitos v1.0 definió once módulos funcionales y doce microservicios preliminares (identidad, pacientes, profesionales, agenda, video, expediente, recetas, pagos, mensajería, notificaciones, integraciones, auditoría), un stack base (Node.js/NestJS, React/Next.js, PostgreSQL, Redis, NATS, mediasoup) y un roadmap en cinco fases. El presente TDD toma esa base como invariante y agrega dos capas arquitectónicas solicitadas por el patrocinador: (1) WSO2 Identity Server sustituye al servicio de autenticación originalmente concebido como microservicio interno, y (2) WSO2 Micro Integrator se introduce como capa de orquestación entre el API Gateway y los microservicios de dominio.

Como consecuencia, el microservicio de identidad del documento previo se restringe a la gestión de perfiles y atributos clínicos, mientras que la autenticación, autorización, gestión de sesiones, MFA y federación pasan a WSO2 IS. El microservicio de integraciones mantiene su rol de fachada FHIR y conector HL7, pero delega al Micro Integrator las transformaciones de mensajes y la composición multi-servicio. El resto de microservicios (pacientes, profesionales, agenda, video, expediente, recetas, pagos, mensajería, notificaciones, auditoría) se conservan con su responsabilidad original.

2.3 Audiencia

El documento está dirigido al equipo técnico y de producto: Tech Lead y desarrolladores full-stack, ingenieros DevOps/SRE, ingeniero de seguridad, arquitecto de soluciones, Product Owner y Compliance Officer. La sección 16 (riesgos) y los ADRs son de interés particular para arquitectura y dirección técnica. Las secciones de identidad (5), APIs (6) y orquestación (7) son de lectura obligatoria para desarrolladores y QA. Las secciones de infraestructura (13) y observabilidad (14) son de interés prioritario para DevOps/SRE.

2.4 Convenciones y notación

Se identifican requisitos técnicos con el código RT-XX (requisito técnico). Los servicios se nombran en español y en kebab-case para los artefactos (p. ej. servicio-expediente). Los componentes de WSO2 se nombran con su producto oficial: WSO2 Identity Server (WSO2 IS), WSO2 API Manager (WSO2 APIM), WSO2 Micro Integrator (WSO2 MI). Los flujos de mediación del Micro Integrator se identifican como SEQ-XX (secuencia). Los eventos asíncronos siguen la especificación CloudEvents y se identifican como EVT-XX. Los endpoints públicos se publican bajo el dominio api.telemedicina.mx y los internos bajo el namespace kube-internal.

2. Decisiones Arquitectónicas (ADRs)

Se registran siete decisiones arquitectónicas significativas (Architecture Decision Records, ADRs) que enmarcan todo el diseño posterior. Cada ADR incluye contexto, decisión, alternativas consideradas y consecuencias. La modificación de cualquiera de ellas requiere un nuevo ADR y la aprobación del comité de arquitectura.

2.1 Lista de decisiones registradas

Las decisiones registradas son: ADR-001, adopción de WSO2 como plataforma unificada para IAM, API management e integración; ADR-002, patrón de microservicios con Node.js/NestJS; ADR-003, PostgreSQL como base relacional principal con Redis y OpenSearch como complementos; ADR-004, NATS como bus de eventos asíncronos; ADR-005, patrón BFF con tres backends para frontend; ADR-006, multi-tenant mediante segregación por columna con row-level security en PostgreSQL; ADR-007, WSO2 Micro Integrator como orquestador de procesos de negocio cross-service.

2.2 ADR-001: Plataforma WSO2 unificada

Contexto: la plataforma requiere un IdP capaz de federar identidades institucionales (SAML/OIDC), aplicar MFA, gestionar consentimiento OAuth para aplicaciones SMART on FHIR, y emitir tokens JWT consumidos por un gateway. Adicionalmente, los flujos de negocio (reserva de cita, pago con CFDI, receta) requieren composición de múltiples servicios con transformaciones, reintentos y compensaciones.

Decisión: adoptar la suite WSO2 en sus tres productos: WSO2 Identity Server 7.x como IdP, WSO2 API Manager 4.x como gateway y portal, y WSO2 Micro Integrator 4.x como orquestador. Los tres componentes se despliegan en el mismo clúster Kubernetes, en un namespace dedicado wso2, y comparten configuración de logging, observabilidad y secretos.

Alternativas consideradas: (a) Keycloak como IdP + Kong como gateway + orquestación ad-hoc en cada servicio; (b) Auth0 + AWS API Gateway + AWS Step Functions; (c) construir el IdP y el gateway como microservicios internos.

Consecuencias: positivo — una sola plataforma coherente, conectores nativos para SAML, OIDC, SCIM, FHIR y SOAP; traza distribuida nativa con OpenTelemetry; cumplimiento HIPAA y RGPD documentado por el proveedor. Negativo — curva de aprendizaje del equipo en

WSO2, costo de licenciamiento comercial o esfuerzo de operación de la versión open source, dependencia de un proveedor para tres capas críticas.

2.3 ADR-002: Microservicios Node.js/NestJS

Contexto: doce servicios de dominio con lógica de negocio heterogénea, escalado independiente y equipos pequeños.

Decisión: cada microservicio se implementa en Node.js 20 LTS con NestJS 11, TypeScript estricto, Prisma ORM, BullMQ para colas locales. La exposición hacia el Micro Integrator es REST/JSON con OpenAPI 3.1; la exposición servicio-a-servicio cuando no media el orquestador puede ser gRPC.

Alternativas: Spring Boot (Java), Go con gRPC-first, Python con FastAPI. Spring Boot se descarta por overhead de runtime para el tamaño de los equipos; Go se descarta por menor ecosistema para FHIR y salud; Python se descarta por restricciones de performance en el servicio de video y agenda.

Consecuencias: positivo — un solo lenguaje end-to-end con el frontend React, ecosistema npm maduro para FHIR (fhir.js, @asymmetrik/fhir-core), contratación de talento accesible. Negativo — Node.js requiere cuidado con CPU-bound (validación de firmas, hashing), que se delega a workers o a servicios especializados.

2.4 ADR-003: PostgreSQL + Redis + OpenSearch

Contexto: datos relacionales transaccionales (citas, pagos, recetas), cache y colas (sesiones, disponibilidad), búsqueda full-text (expedientes, logs).

Decisión: PostgreSQL 16 como base relacional por servicio (database per service), con extensiones pgcrypto, pg_partman y PostGIS. Redis 7 para cache, sesiones y colas BullMQ. OpenSearch 2 para búsqueda y para el store de logs centralizado.

Consecuencias: positivo — modelo transaccional robusto, JSONB para recursos FHIR, replicación síncrona multi-región posible. Negativo — operación de tres stores de datos, necesidad de backups coordinados.

2.5 ADR-004: NATS como bus de eventos

Contexto: comunicación asíncrona entre servicios para eventos de dominio (cita reservada, video finalizado, receta emitida).

Decisión: NATS JetStream como bus de eventos, con subjects jerárquicos (telemedicina.cita.reservada.v1). Los eventos siguen CloudEvents 1.0 en formato JSON.

Alternativas: Kafka (más potente pero más pesado de operar para el volumen esperado), RabbitMQ (cola tradicional, menos orientado a eventos), AWS SNS/SQS (lock-in cloud).

Consecuencias: positivo — operación ligera, at-least-once con JetStream, soporte de request/reply para queries asíncronas. Negativo — sin ordering cross-stream (mitigado con particionado por aggregate id).

2.6 ADR-005: Patrón BFF con tres backends

Contexto: tres clientes frontend (paciente, profesional, administrador) con necesidades de payload y orquestación distintas.

Decisión: tres BFF implementados en Next.js 16 (API routes en modo servidor) que consumen exclusivamente APIs publicadas en WSO2 API Manager. Cada BFF puede invocar varios endpoints del API Gateway y componer respuestas optimizadas para su cliente.

Consecuencias: positivo — los clientes quedan aislados de la topología interna de microservicios, payloads optimizados. Negativo — duplicación de lógica de orquestación entre BFF y Micro Integrator; se resuelve por convención: el BFF orquesta consultas de lectura y presentación, el MI orquesta escrituras y procesos transaccionales.

2.7 ADR-006: Multi-tenant por columna con RLS

Contexto: requisito RF-ADM-02 de segregación estricta por institución en Fase 4, sin costos prohibitivos en Fase 1.

Decisión: cada tabla sensible incluye columna tenant_id; PostgreSQL Row-Level Security (RLS) aplica políticas por tenant basadas en el claim tenant_id del JWT emitido por WSO2 IS. Las bases de datos se comparten entre tenants en Fase 1 y se segregan físicamente para tenants premium en Fase 4 si el volumen lo justifica.

Consecuencias: positivo — operación simplificada en fases tempranas, migración a segregación física sin cambio de código. Negativo — riesgo de fuga cross-tenant si RLS se omite en alguna tabla; mitigado con tests automatizados que validan aislamiento en cada PR.

2.8 ADR-007: WSO2 MI como orquestador

Contexto: los flujos de negocio críticos (reserva de cita con pago diferido, videoconsulta con captura de nota y grabación, emisión de receta con firma y envío, pago con CFDI) requieren coordinar entre tres y seis microservicios con manejo de errores y compensación.

Decisión: el WSO2 Micro Integrator asume el rol de orquestador. Los flujos transaccionales complejos se modelan como secuencias de mediación (proxy services o API con mediación) que invocan a los microservicios de dominio en el orden correcto, aplican transformaciones, persisten estado intermedio en su base de datos interna y ejecutan compensaciones bajo el patrón saga si algún paso falla.

Alternativas: orquestación distribuida en cada servicio con eventos (coreografía pura), o AWS Step Functions. La coreografía pura dificulta la trazabilidad de los procesos y la depuración; Step Functions introduce lock-in cloud.

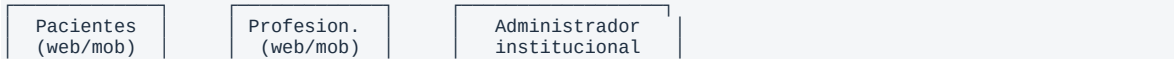
Consecuencias: positivo — visibilidad de los procesos, reintentos y compensaciones centralizados, trazabilidad regulatoria. Negativo — el Micro Integrator se convierte en componente crítico de disponibilidad; se mitiga con despliegue HA y monitoreo dedicado, y con la conservación del patrón saga basado en eventos como respaldo para los flujos más críticos.

3. Visión Arquitectónica

3.1 Diagrama de contexto (C4 nivel 1)

En el nivel de contexto, la Plataforma de Telemedicina es un único sistema que interactúa con seis actores externos. Los pacientes y profesionales acceden vía navegador web o app móvil. Los administradores institucionales acceden vía portal administrativo. Las instituciones cliente federan identidades mediante SAML u OIDC. Los sistemas EHR externos consumen APIs FHIR publicadas en el portal de developers. Los proveedores de servicios auxiliares (Stripe, Mercado Pago, PAC-SAT, WhatsApp Cloud API, Twilio, FCM) reciben llamadas salientes orquestadas por el Micro Integrator. Las autoridades regulatorias (COFEPRIS, INAI, SAT) reciben reportes o comprobantes conforme a la normatividad aplicable.

El siguiente diagrama ASCII sintetiza el contexto. Las flechas indican dirección de invocación principal.



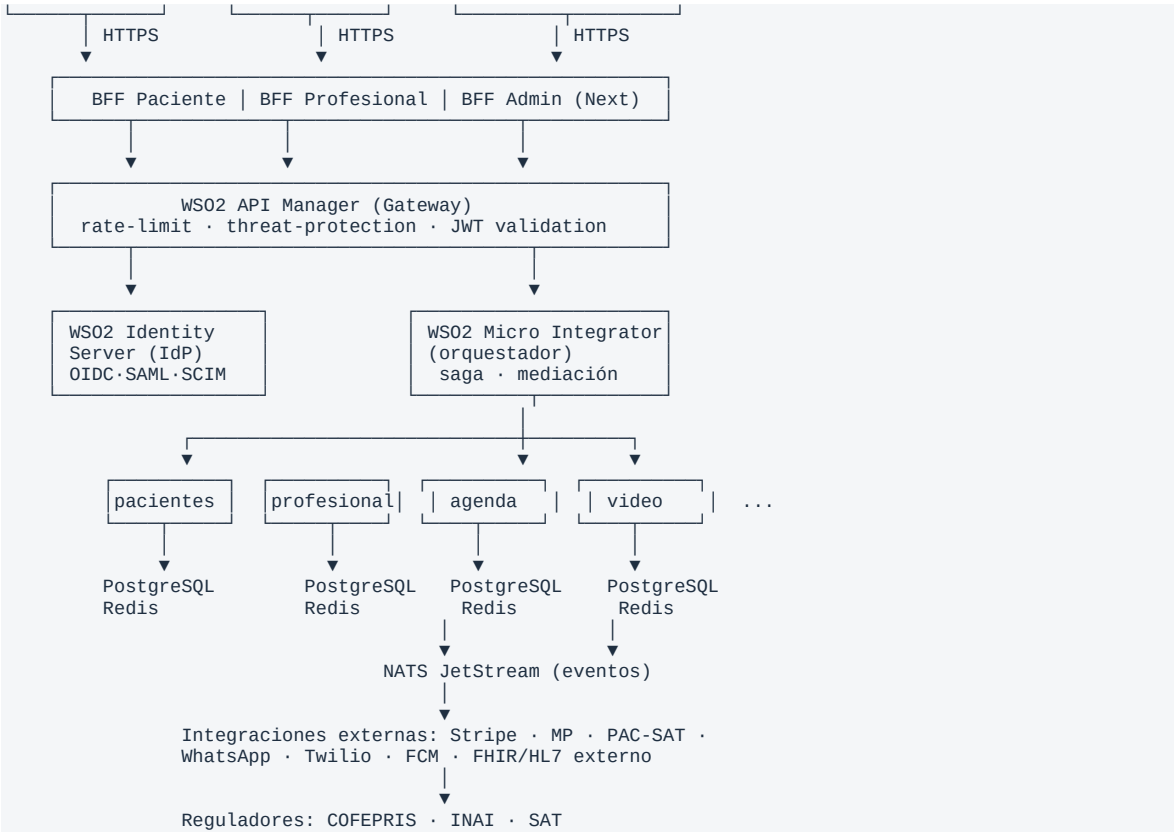


Figura 1. Diagrama de contexto (C4 nivel 1) del sistema.

3.2 Diagrama de contenedores (C4 nivel 2)

El diagrama de contenedores detalla los procesos y stores de datos dentro del sistema. Los contenedores son: tres BFF en Next.js; WSO2 API Manager con sus cuatro sub-componentes (Gateway, Publisher, Dev Portal, Traffic Manager); WSO2 Identity Server; WSO2 Micro Integrator; doce microservicios NestJS; PostgreSQL, Redis y OpenSearch como stores; NATS JetStream como bus; mediasoup como SFU WebRTC; y los pipelines CI/CD basados en GitHub Actions y ArgoCD.

Los flujos de datos principales son: (1) peticiones de cliente → BFF → API Gateway → Micro Integrator → microservicios de dominio → stores; (2) peticiones de autenticación → BFF → API Gateway → WSO2 IS (flujo OIDC Authorization Code con PKCE); (3) eventos de dominio emitidos por microservicios → NATS → suscriptores (otros microservicios, Micro Integrator para sagas); (4) integraciones externas mediadas por Micro Integrator mediante conectores nativos.

3.3 Topología general de red

La red se segmenta en VPC dedicada con subredes públicas (load balancers, WAF, NAT gateway) y privadas (clúster Kubernetes, RDS, Elasticache, OpenSearch). El acceso público entra por un Application Load Balancer tras un WAF (AWS WAF o Cloud Armor) con reglas OWASP. El tráfico se enruta al WSO2 API Manager Gateway, único componente con exposición pública en los puertos 443 (HTTPS) y 8443 (puerto de gestión interno restringido). Los BFF se exponen internamente al ALB mediante Ingress de Kubernetes; el tráfico entre el ALB y los BFF nunca sale de la VPC.

Los microservicios de dominio son inalcanzables desde Internet; solo aceptan tráfico del namespace wso2 y del namespace bff, controlado por NetworkPolicy de Kubernetes. WSO2 IS expone sus endpoints /.well-known/openid-configuration y /oauth2 al API Gateway y opcionalmente a Internet para flujos OAuth directos. WSO2 MI solo expone endpoints internos consumidos por el API Gateway. PostgreSQL, Redis, OpenSearch y NATS residen en subredes privadas sin acceso público, accesibles únicamente desde los pods del clúster mediante security groups.

4. WSO2 Identity Server (Proveedor de Identidad)

WSO2 Identity Server (en adelante WSO2 IS) se adopta como proveedor de identidad centralizado de la plataforma. Es el único componente autorizado para emitir tokens de acceso y de identidad consumidos por el API Gateway y por los microservicios downstream. Toda la lógica de autenticación, federación, MFA, consentimiento y emisión de claims reside en WSO2 IS; los microservicios no implementan lógica de autenticación propia, sino que validan y consumen los claims del JWT emitido por WSO2 IS.

4.1 Rol y responsabilidades

WSO2 IS asume las siguientes responsabilidades, mapeadas a los requisitos del documento de análisis y requisitos v1.0: registro y ciclo de vida de identidades de pacientes, profesionales y administradores (RF-AUTH-01); autenticación multifactor obligatoria para profesionales y administradores, opcional pero recomendada para pacientes (RF-AUTH-02); federación de identidades institucionales mediante SAML 2.0 y OIDC (RF-AUTH-03); gestión de sesiones con expiración configurable y revocación (RF-AUTH-05, RF-AUTH-08); bloqueo temporal tras intentos fallidos mediante políticas de account locking (RF-AUTH-06); recuperación de contraseña con tokens de un solo uso (RF-AUTH-07); y consentimiento OAuth para aplicaciones SMART on FHIR (RF-INT-07).

Adicionalmente, WSO2 IS provee el framework de consentimiento que registra cada autorización del paciente para compartir datos clínicos con aplicaciones externas, cumpliendo los requisitos de RGPD Art. 7 y LFPDPPP. La bitácora de auditoría de identidad se integra al pipeline centralizado de logs y se conserva por el periodo legal aplicable (cinco años mínimo).

4.2 Modelo de identidad

El modelo de identidad distingue tres user stores principales, todos gestionados dentro de una misma instancia de WSO2 IS pero en dominios diferenciados para facilitar la administración y el routing de autenticación.

Dominio TELEMED (usuarios nativos): pacientes registrados directamente en la plataforma. Su identidad se crea por self-service con verificación de email y teléfono (OTP), y se enriquece con claims clínicos: CURP, fecha de nacimiento, sexo, cobertura médica, idioma preferido, representantes legales. Los pacientes pueden tener cuentas vinculadas a representantes (tutores de menores) mediante relaciones jerárquicas gestionadas por WSO2 IS.

Dominio PROF (profesionales): médicos y otros profesionales de la salud. Su identidad se crea solo por invitación o por alta administrativa, no por self-service. Antes de activarse, el microservicio de profesionales valida la cédula ante la DGP y, si corresponde, la habilitación institucional. Solo tras validación exitosa el usuario pasa al rol activo. Los claims clínicos incluyen cédula, especialidad, subespecialidad, institución de adscripción, idiomas y firma electrónica registrada (huella del certificado e.firma).

Dominio ADMIN (administradores): personal interno y de instituciones cliente con acceso al portal administrativo. La creación es exclusivamente por alta de un super-administrador, con MFA obligatoria y rotación de credenciales cada 90 días.

Aplicaciones machine-to-machine: las aplicaciones externas que consumen la API FHIR se registran como service applications con client_credentials grant, sin usuario asociado, y se les asignan scopes limitados a los recursos clínicos que necesitan, previa firma del acuerdo de partner.

4.3 Federación de identidades institucionales

Las instituciones cliente (aseguradoras, hospitales, empresas) que cuenten con su propio IdP pueden federar identidades en lugar de crear cuentas en el dominio TELEMED. WSO2 IS actúa como proveedor de servicio (SP) frente al IdP institucional en SAML 2.0, o como

relying party (RP) en OIDC. La autenticación se delega al IdP institucional, y WSO2 IS recibe las aserciones o tokens, valida la firma, mapea los claims al perfil interno (cédula, institución, rol) y emite su propio JWT para consumo del API Gateway.

Cuando un profesional o paciente federado accede por primera vez, WSO2 IS aplica provisioning just-in-time (JIT) según política: para profesionales, se marca como pendiente de validación hasta que el microservicio de profesionales confirme la cédula; para pacientes institucionales, se omite la verificación OTP si la aserción del IdP institucional ya la contiene y está firmada. La política de provisioning se configura por IdP federado.

Escenario	Protocolo	Rol WSO2 IS	Just-in-time
Hospital con ADFS	SAML 2.0	Service Provider	Sí, con validación de cédula diferida
Aseguradora con Azure AD	OIDC	Relying Party	Sí, marcado como paciente corporativo
Empresa con Okta	SAML 2.0	Service Provider	Sí, plan corporativo asignado
Hospital con WSO2 IS propio	OIDC federation	Federated RP	Sí, claims fusionados

Tabla 1. Escenarios de federación de identidades con WSO2 IS.

4.4 Autenticación multifactor (MFA)

La política de MFA se configura por rol y por dominio. Para profesionales y administradores es obligatoria; para pacientes es opcional pero incentivada (badge en perfil, prioridad en agenda). Los factores soportados son TOTP (Google Authenticator, Authy, Microsoft Authenticator) conforme a RFC 6238; FIDO2/WebAuthn con soporte de llaves YubiKey y platform authenticators (Touch ID, Windows Hello); SMS OTP como factor de recuperación o de segundo factor para usuarios sin acceso a TOTP/FIDO2; y verificación por email OTP para recuperación.

Para entornos sensibles (acceso al expediente completo de un paciente ajeno al propio) se aplica step-up authentication: WSO2 IS evalúa una política adaptativa que solicita un segundo factor en runtime si el acceso implica sensibilidad elevada. Las políticas adaptativas se construyen con el Adaptive Authentication engine de WSO2 IS basado en scripts JavaScript que evalúan claims, contexto (IP, dispositivo, hora) y signals externas (score de riesgo de ThreatMetrix opcional).

Rol	Factor 1	Factor 2 obligatorio	Step-up
Paciente	Password	Opcional (TOTP/FIDO2)	Si accede a expedientes de representados
Profesional	Password	TOTP o FIDO2	Si firma recetas controladas
Administrador	Password	FIDO2 obligatorio	Si cambia configuración global
Service App	Client secret / mTLS	—	—

Tabla 2. Políticas MFA por rol y step-up.

4.5 Autorización: roles, scopes y claims

WSO2 IS emite tokens JWT con claims estandarizados (sub, iss, aud, exp, iat) y claims personalizados que los microservicios consumen para autorización. Los claims clínicos clave son: role (paciente, profesional, administrador), tenant_id (institución), especialidades (array de códigos), cedula (para profesionales), paciente_ids (lista de pacientes que el profesional puede acceder, derivada del módulo profesionales), scope (lista de scopes OAuth solicitados), consent_token (referencia al consentimiento activo para aplicaciones SMART on FHIR).

Los roles se modelan como grupos en WSO2 IS con permisos asociados. La autorización fina (qué expedientes puede ver un profesional) no se resuelve en WSO2 IS sino en el microservicio de expediente, que consulta la relación profesional-paciente y aplica políticas adicionales. WSO2 IS provee la identidad y el rol; el microservicio aplica el ABAC sobre los recursos. Esta separación evita saturar el IdP con reglas de negocio fina.

Los scopes OAuth definen el alcance de acceso. Para aplicaciones internas se usan scopes funcionales (cita:write, expediente:read, receta:write). Para aplicaciones SMART on FHIR externas se usan scopes clínicos (patient/Observation.read, patient/MedicationRequest.write) con consentimiento explícito del paciente registrado por el framework de consentimiento de WSO2 IS.

Scope	Concedido a	Recurso	Consentimiento
cita:write	Paciente, Profesional	Citas propias/asignadas	Implícito al login
expediente:read	Profesional	Pacientes bajo cuidado	Verificado en servicio
receta:write	Profesional verificado	Sus pacientes	MFA step-up

Scope	Concedido a	Recurso	Consentimiento
patient/Observation.read	App SMART on FHIR	Observaciones del paciente	Explícito OAuth
admin:tenant:config	Administrador	Config tenant propio	FIDO2

Tabla 3. Catálogo de scopes OAuth y consentimiento.

4.6 Provisioning con SCIM 2.0

El alta, modificación y baja de identidades entre WSO2 IS y los microservicios se realiza mediante SCIM 2.0 (System for Cross-domain Identity Management). WSO2 IS actúa como proveedor SCIM y los microservicios de pacientes y profesionales como consumidores. Cuando un paciente completa el registro, WSO2 IS emite un evento SCIM user.created que el microservicio de pacientes consume vía webhook para crear el perfil clínico local con el user_id de WSO2 IS como clave externa.

Para profesionales, el flujo es inverso: el alta administrativa en el microservicio de profesionales dispara una llamada SCIM user.create hacia WSO2 IS, que crea la identidad y devuelve el user_id; el profesional recibe entonces un email de activación con enlace al flujo de set-password y MFA enrollment. La sincronización de atributos (cambio de especialidad, baja por sanción) usa PATCH SCIM hacia WSO2 IS, que actualiza los claims disponibles en tokens futuros.

4.7 Consentimiento para aplicaciones SMART on FHIR

Cuando una aplicación externa SMART on FHIR solicita acceso a los datos clínicos de un paciente, WSO2 IS ejecuta el flujo OAuth 2.0 Authorization Code con PKCE y presenta al paciente la pantalla de consentimiento, que detalla los scopes solicitados (patient/Observation.read, patient/MedicationRequest.write, etc.), el nombre de la aplicación, el propósito declarado y la duración del acceso. El paciente puede aprobar o rechazar granularmente por scope.

El consentimiento se persiste en el store de consentimiento de WSO2 IS y se referencia en cada token JWT emitido para esa aplicación mediante el claim consent_token. El microservicio de integraciones FHIR valida el consent_token antes de servir cada recurso, y revoca el acceso si el consentimiento fue retirado. El paciente puede auditar y revocar consentimientos desde su perfil, conforme a los derechos ARSO del RGPD y los derechos ARCO de la LFPDPPP.

4.8 Cumplimiento regulatorio de la identidad

WSO2 IS está certificado para los perfiles OIDC Basic, Implicit, Hybrid y Config; soporta FAPI 1.0 Baseline y Advanced para flujos financieros-equivalentes, aplicables a recetas de controlados; cumple con las guías NIST 800-63B de autenticación digital; y tiene assessment de cumplimiento HIPAA y RGPD documentado por el proveedor. La configuración del IdP debe alinearse con los siguientes requisitos: tokens de acceso con vida útil máxima de 15 minutos; refresh tokens con rotación y vida útil de 30 días; subject identifier pairwise para aplicaciones SMART on FHIR (sub distinto por cliente, evitando correlación); almacenamiento de contraseñas con PBKDF2 o Argon2; y bitácora de auditoría inmutable de eventos de identidad.

5. WSO2 API Manager (Gateway y Portal)

WSO2 API Manager (en adelante WSO2 APIM) es el gateway de APIs y el portal de publicación. Acepta exclusivamente tráfico con token JWT emitido por WSO2 IS, valida la firma y los claims, aplica políticas de throttling y seguridad, enruta al Micro Integrator o directamente al microservicio correspondiente, y expone un portal de developers con documentación OpenAPI 3.1 autogenerada.

5.1 Componentes del producto

WSO2 APIM se compone de cuatro sub-componentes que se despliegan de forma independiente para escalado selectivo. El API Gateway es el componente de runtime que valida tokens, aplica políticas y enruta; debe escalarse horizontalmente y residir lo más cerca posible del Micro Integrator para minimizar latencia. El Publisher es el portal interno donde los diseñadores de API publican nuevas APIs, definen políticas y gestionan el ciclo de vida; es de uso interno del equipo de plataforma. El Dev Portal (Developer Portal) es el portal público donde los integradores externos descubren APIs, se suscriben, obtienen claves y leen la documentación; se publica en el dominio developers.telemedicina.mx. El Traffic Manager es el componente de control de cuotas y rate limiting en tiempo real, basado en un store Redis compartido.

Componente	Rol	Escala	Exposición
API Gateway	Valida JWT, aplica políticas, enruta	Horizontal 3+ réplicas	Privada, detrás del ALB

Componente	Rol	Escala	Exposición
Publisher	Publicación y lifecycle de APIs	Single o HA	Interna
Dev Portal	Catálogo público y suscripciones	Horizontal 2+	Pública developers.telemedicina.mx
Traffic Manager	Cuotas y throttling global	HA 2 réplicas	Interna

Tabla 4. Componentes de WSO2 API Manager.

5.2 Catálogo de APIs publicadas

El catálogo inicial publica las siguientes APIs, organizadas en tres grupos de audiencia: APIs internas para los BFF, APIs externas para aplicaciones SMART on FHIR, y APIs administrativas para los portales institucionales. Cada API expone una versión v1 y se versiona con la política semver: cambios breaking generan v2, coexistiendo hasta deprecación.

API	Audiencia	Versión	Backend (orquestador)	Scopes típicos
Paciente API	BFF Paciente	v1	WSO2 MI → pacientes + agenda	cita:write, expediente:read:self
Profesional API	BFF Profesional	v1	WSO2 MI → profesionales + agenda	cita:write, expediente:read
Video API	BFF (pac+prof)	v1	Directo a video service	video:session
Receta API	BFF Profesional	v1	WSO2 MI → recetas + expediente	receta:write
Pago API	BFF Paciente	v1	WSO2 MI → pagos + PAC-SAT	pago:write
Mensaje API	BFF (pac+prof)	v1	Directo a mensajería	mensaje:write
FHIR R4 API	Apps externas SMART	v1	WSO2 MI → integraciones + expediente	patient/*.read
Admin API	BFF Admin	v1	Directo a administración	admin:tenant:*
Webhooks API	Apps externas	v1	WSO2 MI → notificaciones	webhook:register

Tabla 5. Catálogo inicial de APIs publicadas.

5.3 Políticas de seguridad y throttling

El API Gateway aplica las siguientes políticas por defecto: validación de firma JWT con clave pública de WSO2 IS publicada en JWKS endpoint; validación de expiración y audience; validación de scopes requeridos por endpoint; verificación de revocación mediante introspection endpoint cacheado con TTL de 30 segundos; rate limiting por tier (bronze 10 req/min, silver 60, gold 300, platinum 1000); threat protection con tamaño máximo de payload 1 MB y validación de JSON schema; cabeceras de seguridad (HSTS, X-Content-Type-Options, X-Frame-Options: DENY, CSP restrictiva); CORS configurado por API con orígenes permitidos explícitos.

Tier	Cuota	Burst	Audiencia	Costo
Bronze	10 req/min	20	Apps free tier	Sin costo
Silver	60 req/min	120	Apps profesionales	Suscripción
Gold	300 req/min	600	Instituciones cliente	Por usuario
Platinum	1000 req/min	2000	Integraciones EHR	Contrato
Internal	Sin límite	—	BFF (mismo clúster)	—

Tabla 6. Tiers de throttling por audiencia.

5.4 Versionado y ciclo de vida de APIs

Cada API pasa por los estados CREATED, PUBLISHED, DEPRECATED, RETIRED. La transición PUBLISHED → DEPRECATED ocurre cuando se publica una versión mayor nueva; los suscriptores son notificados por email y por banner en el Dev Portal. El estado RETIRED aplica tras un periodo mínimo de 6 meses en DEPRECATED, durante el cual los endpoints responden 410 Gone con cabecera Sunset. Los cambios no breaking (nuevos endpoints, nuevos campos opcionales) se publican como minor version dentro de la misma versión mayor, sin deprecación.

5.5 Portal de developers

El Dev Portal expone: catálogo de APIs con documentación OpenAPI 3.1 interactiva (Try it out en sandbox); suscripción con generación de claves client_id/client_secret; panel de uso con métricas por API y por aplicación; foro de soporte integrado; SDKs descargables en JavaScript, Python y PHP generados automáticamente desde OpenAPI; ejemplos en tres lenguajes por endpoint; changelog por versión; y guía de onboarding SMART on FHIR para aplicaciones clínicas externas. El branding del portal es personalizable por tenant en Fase 4.

6. WSO2 Micro Integrator (Orquestación por Servicios)

WSO2 Micro Integrator (en adelante WSO2 MI) es el componente que orquesta la integración entre servicios. Asume el rol de mediador entre el API Gateway y los microservicios de dominio para todos los flujos que requieren composición de múltiples servicios, transformación de mensajes, integración con sistemas externos o manejo de errores con compensación. Los microservicios exponen APIs REST internas estables; WSO2 MI las consume, las combina y expone al API Gateway endpoints cohesivos que corresponden a procesos de negocio.

6.1 Rol del orquestador

El orquestador tiene cuatro responsabilidades núcleo. Primera, composición: un endpoint del API Gateway puede requerir información de dos o más microservicios (p. ej. obtener el detalle de una cita implica agenda, pacientes, profesionales y pagos); WSO2 MI ejecuta las llamadas en paralelo cuando no hay dependencia, en secuencia cuando la hay, y compone la respuesta. Segunda, transformación: los microservicios usan su modelo de datos interno, mientras que las APIs expuestas usan un modelo canónico (p. ej. recursos FHIR para la API externa); WSO2 MI aplica las transformaciones con Data Mapper o XSLT según el conector.

Tercera, integración con sistemas externos: las llamadas a Stripe, Mercado Pago, PAC-SAT, WhatsApp Cloud API, Twilio, FCM, EHR externos vía FHIR y HL7 v2 se realizan mediante los conectores nativos de WSO2 MI, evitando reimplementarlos en cada microservicio. Cuarta, manejo de procesos: los flujos transaccionales multi-paso que cruzan servicios se modelan como sagas coordinadas por WSO2 MI, con estado intermedio persistido en su base de datos interna y compensaciones ejecutadas si algún paso falla.

6.2 Topología de integración por servicios

WSO2 MI se despliega como un cluster de instancias stateless detrás del API Gateway, con estado transaccional persistido en una base de datos compartida (MySQL o PostgreSQL dedicado del namespace wso2). Cada flujo orquestado se publica como un API proxy en WSO2 APIM, que enruta al endpoint del MI correspondiente. Los flujos internos del MI se implementan como secuencias de mediación (mediation sequences) en XML o DSL de WSO2, versionadas en Git junto con el resto del código.

Para integraciones salientes hacia sistemas externos, WSO2 MI expone los conectores como artifacts reutilizables: el conector Stripe, el conector Mercado Pago, el conector FHIR, el conector HTTP genérico para WhatsApp/Twilio/FCM. Cada conector encapsula

autenticación, reintentos, time-to-live y mapeo de errores. Los secretos de los conectores se inyectan desde el gestor de secretos del clúster (Vault o AWS Secrets Manager) mediante variables de entorno.

Componente	Instancias	Persistencia	Latencia objetivo
WSO2 MI runtime	3+ stateless	MySQL/PG compartido para sagas	< 50 ms overhead
Connectors	Embebidos en MI	—	Variable por conector
Message stores	Persistentes para retry	MySQL del namespace wso2	—

Tabla 7. Topología de componentes del Micro Integrator.

6.3 Patrones de integración aplicados

Se aplican cinco patrones de integración canónicos. Patrón API Proxy: el MI expone un endpoint que enruta a un único microservicio sin transformación, útil para separar la versión pública de la interna. Patrón Aggregator: el MI llama en paralelo a varios servicios y compone la respuesta (p. ej. GET /citas/{id}/detalle llama a agenda, pacientes, profesionales, pagos en paralelo). Patrón Sequential: el MI llama a servicios en secuencia cuando cada paso depende del anterior (p. ej. receta: validar cédula → emitir receta → firmar → notificar).

Patrón Saga coordinada: el MI coordina un proceso multi-paso transaccional con compensación (p. ej. pago con CFDI: crear cargo en Stripe → generar CFDI en PAC → actualizar pago en servicio → emitir receta si aplica → notificar). Patrón Broker: el MI publica eventos en NATS y consume eventos para disparar flujos asíncronos (p. ej. al recibir evento cita.reservada, orquesta el envío de confirmación por email/SMS/WhatsApp).

La elección entre sincrónico y asíncrono depende del SLA: las consultas de lectura son sincrónicas; los procesos de escritura complejos con sistemas externos lentos (PAC-SAT puede tardar 5-10 segundos) son asíncronos con polling o webhook de notificación al cliente.

Patrón	Cuándo se aplica	Ejemplo concreto
API Proxy	Sin transformación, solo enrutamiento	GET /admin/users → admin service
Aggregator paralelo	Varias fuentes independientes	GET /citas/{id}/detalle
Sequential	Pasos dependientes	POST /recetas (validar→emitir→firmar)

Patrón	Cuándo se aplica	Ejemplo concreto
Saga coordinada	Multi-paso transaccional con compensación	POST /pagos/{id}/capturar
Broker evento	Reacción a eventos asíncronos	cita.reservada → notificaciones multi-canal

Tabla 8. Patrones de integración canónicos aplicados.

6.4 Secuencias de mediación principales

Se identifican ocho secuencias de mediación principales, cada una modelada como una saga o aggregator según corresponda. Cada secuencia se identifica como SEQ-XX y se versiona en Git. Las secuencias se detallan en la sección 11 con diagramas de secuencia; aquí se enuncian.

ID	Secuencia	Servicios involucrados	Tipo
SEQ-01	Reserva de cita con autorización diferida	agenda + pacientes + profesionales + pagos (auth only) + notificaciones	Saga
SEQ-02	Check-in y admisión a videoconsulta	agenda + video + notificaciones	Sequential
SEQ-03	Cierre de videoconsulta con captura de nota	video + expediente + agenda + notificaciones	Saga
SEQ-04	Emisión de receta electrónica firmada	recetas + profesionales (validación) + expediente + PAC firma + notificaciones	Saga
SEQ-05	Captura de pago con CFDI	pagos + Stripe/MP + PAC-SAT + expediente (referencia) + notificaciones	Saga
SEQ-06	Reembolso total o parcial	pagos + Stripe/MP + PAC-SAT (nota crédito) + notificaciones	Saga
SEQ-07	Exportación FHIR del expediente	integraciones + expediente + pacientes + profesionales	Aggregator
SEQ-08	Interconsulta con consentimiento	mensajería + expediente (compartir) + notificaciones	Sequential

Tabla 9. Catálogo de secuencias de mediación (SEQ-01 a SEQ-08).

6.5 Conectores externos

WSO2 MI mantiene un catálogo de conectores externos reutilizables. Cada conector encapsula autenticación, manejo de errores y mapeo de respuesta. La gestión de secretos

se realiza mediante el integration con Vault: el MI recupera los secretos al arranque y los mantiene en memoria con rotación periódica. Los conectores externos identificados son: Stripe (pagos internacionales, tokenización PCI-DSS), Mercado Pago (pagos LatAm, SPEI), PAC-SAT (CFDI 4.0, timbrado), WhatsApp Cloud API (mensajería plantillas), Twilio (SMS y voz), Firebase Cloud Messaging (push móvil), Web Push (push navegador), e.firma/SAT (firma electrónica avanzada), DGP (validación de cédulas), FHIR R4 (conector hacia EHR externos), HL7 v2 (parser y mapeo a FHIR), DICOM/PACS (conector a sistemas de imágenes).

Conector	Sistema	Versión	Reintentos	Circuito
stripe-connector	Stripe API	2024-06-20	3 exp backoff	Sí
mercadopago-connector	MP API	v1	3 exp backoff	Sí
sat-cfdi-connector	PAC autorizado	CFDI 4.0	5 intentos lineales	Sí
whatsapp-cloud-connector	Meta Cloud API	v18	2 exp backoff	Sí
twilio-connector	Twilio API	2010-04-01	3 exp backoff	Sí
fhir-connector	HL7 FHIR	R4	2 lineales	No
hl7v2-connector	HL7 v2	2.5.1	2 lineales	No

Tabla 10. Catálogo de conectores externos del Micro Integrator.

6.6 Manejo de errores, reintentos y compensación

Toda secuencia de mediación implementa un manejo de errores uniforme. Los errores transitorios (timeout 5xx, 429) se reintentan con backoff exponencial hasta el límite configurado por conector. Los errores de negocio (400, 422) no se reintentan y se propagan al cliente con código y mensaje canónicos. Los errores fatales (caída del MI, fallo de un servicio downstream en una saga) disparan la secuencia de compensación: el MI consulta su tabla de saga_state, identifica los pasos ya completados y ejecuta las compensaciones en orden inverso.

Cada saga define su tabla de compensaciones al diseñarse. Por ejemplo, en SEQ-05 (pago con CFDI), si la generación del CFDI falla tras capturar el pago en Stripe, la compensación ejecuta el reembolso en Stripe y marca el pago como revertido. Si la notificación al paciente falla tras la generación exitosa del CFDI, no se compensa (la transacción principal ya quedó confirmada); el evento fallido se encola en NATS para reintento asíncrono por el microservicio de notificaciones, evitando degradar el tiempo de respuesta del cliente.

6.7 Trazabilidad de los flujos orquestados

Cada invocación al API Gateway recibe un correlation_id (UUID v4) propagado como cabecera X-Correlation-Id. WSO2 APIM lo inyecta si no viene del cliente. WSO2 MI propaga la cabecera a todas las llamadas a microservicios y a los conectores externos. Los microservicios incluyen el correlation_id en todos los logs estructurados y en los eventos publicados en NATS (campo source en CloudEvents). OpenTelemetry traceparent también se propaga para trazas distribuidas. Esto permite, ante cualquier incidente clínico o auditoría, reconstruir el flujo completo desde el gateway hasta cada servicio y conector externo, requisito explícito de la NOM-024 para trazabilidad de operaciones sobre el expediente clínico electrónico.

7. Catálogo de Microservicios de Dominio

Se detallan los doce microservicios de dominio. Cada uno es autónomo, con su propia base de datos PostgreSQL, su cache Redis dedicado y sus colas BullMQ cuando aplica. La comunicación con el exterior se realiza exclusivamente a través del API Gateway (mediado o no por WSO2 MI); la comunicación directa servicio-a-servicio se minimiza y, cuando ocurre, es por gRPC dentro del mismo clúster con mTLS. Cada servicio publica eventos de dominio en NATS JetStream conforme a la especificación CloudEvents.

8.1 Servicio Identidad (servicio-identidad)

Responsabilidad: Gestión de perfiles de identidad y atributos clínicos locales; puente con WSO2 IS mediante SCIM 2.0. La autenticación, MFA y federación las realiza WSO2 IS; este servicio mantiene la asociación user_id (WSO2) ↔ perfil interno y los claims clínicos extendidos.

Base de datos: PostgreSQL (identidad_db): tablas usuario_perfil, usuario_claims, sesiones_historicas, dispositivos.

Eventos publicados: identidad.usuario.creado.v1, identidad.usuario.actualizado.v1, identidad.usuario.bloqueado.v1

Eventos consumidos: identidad.usuario.creado.v1 (de WSO2 IS por webhook)

Dependencias: WSO2 IS (SCIM, webhooks), DGP (validación cédulas), Notificaciones (vía NATS)

8.2 Servicio Pacientes (servicio-pacientes)

Responsabilidad: Perfil demográfico completo del paciente: CURP, RFC, datos contacto, antecedentes editables por el paciente, representantes legales, coberturas médicas, preferencias, consentimientos informados digitales.

Base de datos: PostgreSQL (pacientes_db): paciente, antecedente, representante, cobertura, consentimiento.

Eventos publicados: paciente.creado.v1, paciente.actualizado.v1, paciente.consentimiento.otorgado.v1

Eventos consumidos: identidad.usuario.creado.v1

Dependencias: Identidad (vía NATS), Notificaciones

8.3 Servicio Profesionales (servicio-profesionales)

Responsabilidad: Perfil profesional, especialidades, tarifas, calendario de disponibilidad, afiliaciones institucionales, certificados cargables, reputación. Realiza validación de cédula ante DGP (vía WSO2 MI).

Base de datos: PostgreSQL (profesionales_db): profesional, especialidad, tarifa, disponibilidad, institucion_afiliacion.

Eventos publicados: profesional.creado.v1, profesional.verificado.v1, disponibilidad.actualizada.v1

Eventos consumidos: identidad.usuario.creado.v1

Dependencias: Identidad, DGP (vía MI)

8.4 Servicio Agenda y Citas (servicio-agenda)

Responsabilidad: Ciclo completo de citas: reserva, confirmación, reprogramación, cancelación, recordatorios, sala de espera virtual, no-shows. Motor de reglas por institución, médico y tipo de consulta.

Base de datos: PostgreSQL (agenda_db): cita, slot, sala_espera, no_show, politica_cancelacion.

Eventos publicados: cita.reservada.v1, cita.confirmada.v1, cita.cancelada.v1, cita.reprogramada.v1, cita.checkin.v1, cita.noshow.v1

Eventos consumidos: disponibilidad.actualizada.v1, profesional.verificado.v1

Dependencias: Pacientes, Profesionales, Pagos (auth), Notificaciones (vía NATS)

8.5 Servicio Videoconsulta (servicio-video)

Responsabilidad: Gestión de sesiones WebRTC con SFU mediasoup: creación de room, generación de tokens de acceso, manejo de grabación con consentimiento, reconexión automática, chat paralelo, marcadores clínicos.

Base de datos: PostgreSQL (video_db): sesion_video, room, participante, grabacion, marcador_clinico. Object storage S3 para grabaciones.

Eventos publicados: video.sesion.iniciada.v1, video.sesion.finalizada.v1, video.grabacion.disponible.v1

Eventos consumidos: cita.checkin.v1, cita.noshow.v1

Dependencias: Agenda, Expediente (vía MI para marcadores), Notificaciones

8.6 Servicio Expediente Clínico (servicio-expediente)

Responsabilidad: Núcleo clínico conforme NOM-024: notas de evolución, signos vitales, antecedentes estructurados (SNOMED CT), medicamentos activos con interacciones, adjuntos multimedia, marca temporal y autoría inmutable, exportación FHIR.

Base de datos: PostgreSQL (expediente_db) con particionado temporal por paciente. Object storage para adjuntos. Search index en OpenSearch.

Eventos publicados: expediente.nota.creada.v1, expediente.signo_vital.registrado.v1, expediente.adjunto.cargado.v1, expediente.acceso.registrado.v1

Eventos consumidos: video.sesion.finalizada.v1, cita.checkin.v1

Dependencias: Pacientes, Profesionales, Auditoría

8.7 Servicio Recetas (servicio-recetas)

Responsabilidad: Generación de recetas electrónicas con e.firma avanzada del médico, manejo de medicamentos controlados con receta especial, código QR anti-falsificación, envío al paciente y a farmacias asociadas, reportes a COFEPRIS para controlados.

Base de datos: PostgreSQL (recetas_db): receta, item_receta, firma_digital, envio, qr_verificacion.

Eventos publicados: receta.emitida.v1, receta.firmada.v1, receta.enviada.v1

Eventos consumidos: expediente.nota.creada.v1

Dependencias: Profesionales (validación), e.firma/SAT (vía MI), Farmacias asociadas (vía MI)

8.8 Servicio Pagos y Facturación (servicio-pagos)

Responsabilidad: Procesa pagos vía Stripe y Mercado Pago con tokenización PCI-DSS; autorización y captura diferida; facturación CFDI 4.0 vía PAC autorizado por SAT; reembolsos; conciliación bancaria; facturación consolidada por institución.

Base de datos: PostgreSQL (pagos_db): transaccion, autorizacion, captura, factura_cfdi, reembolso, conciliacion.

Eventos publicados: pago.autorizado.v1, pago.capturado.v1, pago.reembolsado.v1, cfdi.generado.v1

Eventos consumidos: cita.reservada.v1, cita.noshow.v1

Dependencias: Stripe (vía MI), Mercado Pago (vía MI), PAC-SAT (vía MI), Notificaciones

8.9 Servicio Mensajería (servicio-mensajería)

Responsabilidad: Chat asíncrono cifrado paciente-médico, interconsultas entre profesionales con adjuntos, conservación de mensajes clínicos relacionados con un caso.

Base de datos: PostgreSQL (mensajeria_db): conversacion, mensaje, adjunto_mensaje, interconsulta.

Eventos publicados: mensaje.enviado.v1, interconsulta.solicitada.v1

Eventos consumidos: cita.reservada.v1

Dependencias: Expediente (archivado de mensajes clínicos), Notificaciones

8.10 Servicio Notificaciones (servicio-notificaciones)

Responsabilidad: Centro de notificaciones multi-canal: push web/móvil (FCM), SMS (Twilio), WhatsApp (Cloud API), email (SES/Sendgrid). Gestiona preferencias por usuario, escalamiento de alertas críticas, plantillas aprobadas.

Base de datos: PostgreSQL (notificaciones_db): notificacion, plantilla, preferencia, intento_envio.

Eventos publicados: notificacion.enviada.v1, notificacion.fallida.v1

Eventos consumidos: cita.reservada.v1, cita.confirmada.v1, cita.cancelada.v1, receta.emitida.v1, expediente.nota.creada.v1, pago.capturado.v1, alerta.critica.laboratorio.v1

Dependencias: FCM (vía MI), Twilio (vía MI), WhatsApp (vía MI), Email provider

8.11 Servicio Integraciones EHR (servicio-integraciones)

Responsabilidad: Fachada FHIR R4 hacia aplicaciones externas y conector HL7 v2 hacia sistemas legacy. Portal de developers con OpenAPI. SDKs oficiales. Webhooks para eventos clínicos. OAuth SMART on FHIR con consentimiento.

Base de datos: PostgreSQL (integraciones_db): aplicacion, clave_api, webhook, evento_suscripcion, fhir_resource_cache.

Eventos publicados: integracion.webhook.entregado.v1, integracion.fhir.solicitado.v1

Eventos consumidos: expediente.nota.creada.v1, expediente.signo_vital.registrado.v1, receta.emitida.v1

Dependencias: Expediente (vía MI para composition FHIR), WSO2 IS (consentimiento)

8.12 Servicio Auditoría (servicio-auditoria)

Responsabilidad: Bitácora inmutable de auditoría: cada acceso, creación, modificación o eliminación de datos sensibles genera entrada. Append-only con hash encadenado, retención configurable por jurisdicción.

Base de datos: PostgreSQL (auditoria_db) con particionado temporal pg_partman. Tabla append-only: evento_auditoria con hash_previo, hash_actual, payload_cifrado.

Eventos publicados: auditoria.evento.registrado.v1

Eventos consumidos: expediente.acceso.registrado.v1, identidad.usuario.bloqueado.v1, pago.capturado.v1, receta.firmada.v1, video.sesion.iniciada.v1

Dependencias: Todos los servicios (vía NATS)

8. Modelo de Datos

Cada microservicio es dueño de su base de datos (database per service). No hay acceso cruzado directo a bases entre servicios; la composición se realiza vía API mediada por WSO2

MI. A continuación se describen las entidades principales por servicio y el mapeo a recursos FHIR R4, base de la interoperabilidad hacia sistemas externos.

8.1 Bases de datos por servicio

Cada servicio posee una base PostgreSQL dedicada en una instancia RDS multi-AZ con backups automáticos cada 15 minutos y snapshot diario retenido 30 días. Los servicios con alta carga de lectura (expediente, agenda) replican lecturas a réplicas síncronas. Redis se aprovisiona por servicio en instancias Elasticache separadas para evitar interferencia. OpenSearch se comparte entre expediente y auditoría con índices dedicados.

Servicio	Base PG	Redis	OpenSearch	Object Store
Identidad	identidad_db	Sí (cache claims)	—	—
Pacientes	pacientes_db	Sí (perfil)	—	—
Profesionales	profesionales_db	Sí (perfil público)	Sí (búsqueda)	—
Agenda	agenda_db	Sí (slots)	—	—
Video	video_db	Sí (tokens)	—	S3 (grabaciones)
Expediente	expediente_db	Sí (cache paciente)	Sí (full-text)	S3 (adjuntos)
Recetas	recetas_db	—	—	—
Pagos	pagos_db	—	—	—
Mensajería	mensajeria_db	Sí (conversaciones activas)	—	S3 (adjuntos)
Notificaciones	notificaciones_db	Sí (colas)	—	—
Integraciones	integraciones_db	Sí (cache FHIR)	—	—
Auditoría	auditoria_db	—	Sí (logs central)	—

Tabla 11. Stores de datos por microservicio.

8.2 Entidades principales por servicio

Se listan las entidades críticas de cada servicio con su clave primaria y las relaciones más importantes. El detalle de columnas, índices y constraints se entrega en el modelo físico Prisma de cada repositorio.

Servicio	Entidad	PK	Relaciones clave
Identidad	usuario_perfil	user_id (UUID)	1-N claims, 1-N sesiones
Pacientes	paciente	paciente_id	1-N antecedentes, 1-N consentimientos, 1-N representantes
Profesionales	profesional	profesional_id	1-N especialidades, 1-N tarifas, 1-N disponibilidades
Agenda	cita	cita_id	N-1 paciente, N-1 profesional, 1-1 pago, 1-1 video_sesion
Video	sesion_video	sesion_id	1-1 cita, 1-N participantes, 0-N grabaciones, 0-N marcadores
Expediente	nota_evolucion	nota_id	N-1 paciente, N-1 profesional, 1-N adjuntos, 1-N signos_vitales
Recetas	receta	receta_id	N-1 paciente, N-1 profesional, 1-N items, 1-1 firma, 0-N envios
Pagos	transaccion	transaccion_id	1-1 cita, 0-N capturas, 0-N reembolsos, 1-N facturas_cfdi
Mensajería	conversacion	conversacion_id	N-1 cita (opcional), 2-N participantes, 1-N mensajes
Notificaciones	notificacion	notificacion_id	N-1 destinatario, 1-N intentos_envio
Integraciones	aplicacion	app_id	1-N claves, 1-N webhooks, 1-N suscripciones_evento
Auditoría	evento_auditoria	evento_id	Auto-referencia hash_previo → evento anterior

Tabla 12. Entidades principales y relaciones por servicio.

8.3 Mapeo a FHIR R4

La plataforma usa FHIR R4 como modelo canónico para la API externa y para la exportación conforme a NOM-024. El microservicio de integraciones expone los siguientes recursos FHIR, poblados por composición orquestada por WSO2 MI a partir de datos de los servicios internos.

Recurso FHIR	Origen interno	Servicio	Notas
Patient	paciente	Pacientes	Identificadores CURP como identifier
Practitioner	profesional	Profesionales	cédula como identifier

Recurso FHIR	Origen interno	Servicio	Notas
PractitionerRole	institucion_afiliacion	Profesionales	Rol por institución
Appointment	cita	Agenda	status mapeado a FHIR
Encounter	cita + video_sesion	Agenda+Video	Encuentro virtual
Observation	signo_vital	Expediente	LOINC codes
ClinicalImpression	nota_evolucion	Expediente	Notas estructuradas
MedicationRequest	receta + item_receta	Recetas	Recetas como MedicationRequest
DocumentReference	adjunto	Expediente	Adjuntos del expediente
Communication	mensaje	Mensajería	Mensajes clínicos
AuditEvent	evento_auditoria	Auditoría	Bitácora FHIR
Consent	consentimiento	Pacientes	Consentimientos informados

Tabla 13. Mapeo de entidades internas a recursos FHIR R4.

9. Comunicación y Eventos

La comunicación entre componentes sigue tres modalidades según el caso: sincrónica vía REST a través del API Gateway y WSO2 MI para flujos cliente; directa servicio-a-servicio vía gRPC con mTLS solo cuando el orquestador no justifica overhead (lecturas internas, salud de servicio); y asíncrona vía NATS JetStream para eventos de dominio. La elección se rige por la necesidad de respuesta inmediata del cliente y por la naturaleza transaccional del flujo.

9.1 Comunicación sincrónica

Las peticiones de los clientes (BFF) se dirigen al API Gateway de WSO2 APIM, que valida el JWT emitido por WSO2 IS, aplica políticas de throttling y threat protection, y enruta al endpoint expuesto por WSO2 MI (para flujos orquestados) o directamente al microservicio (para endpoints simples proxy). Los microservicios exponen APIs REST/JSON documentadas con OpenAPI 3.1; las llamadas internas del MI a los microservicios usan mTLS con certificados gestionados por cert-manager del clúster.

9.2 Comunicación asíncrona con NATS JetStream

Los eventos de dominio se publican en NATS JetStream con subjects jerárquicos: telemedicina.<aggregate>.<event>.v<n>. Por ejemplo telemedicina.cita.reservada.v1. JetStream garantiza persistencia duradera y at-least-once delivery. Los suscriptores implementan idempotencia mediante el campo id del evento (UUID generado por el publicador) y una tabla de eventos_procesados en su base de datos.

9.3 Formato CloudEvents

Todos los eventos siguen la especificación CloudEvents 1.0 en formato JSON. Los campos obligatorios son specversion, id, source, type, time, datacontenttype, data. Adicionalmente se incluye correlation_id en una extensión para propagar la trazabilidad desde el API Gateway. El campo data contiene el payload específico del evento, normalmente el aggregate en estado post-evento o el delta producido.

9.4 Patrón outbox para publicación transaccional

Para evitar la inconsistencia entre commit de base de datos y publicación de evento (problema de dual-write), los microservicios implementan el patrón outbox. Cada transacción de negocio que produce un evento inserta, en la misma transacción PostgreSQL, una fila en la tabla outbox con el evento. Un proceso worker (BullMQ) lee la tabla outbox, publica en NATS y marca como enviada. Esto garantiza que el evento se publique si y solo si la transacción de negocio cometió.

9.5 Patrón saga coordinada por WSO2 MI

Los flujos transaccionales que cruzan servicios se orquestan como sagas coordinadas por WSO2 MI. Cada paso de la saga se persiste en la tabla saga_state del namespace wso2. Si un paso falla de forma no recuperable, el MI ejecuta las compensaciones en orden inverso para los pasos ya completados. La comunicación entre el MI y los servicios participantes es sincrónica (REST) dentro de cada paso; la coordinación entre pasos es responsabilidad del MI, no de eventos. Esta elección privilegia la trazabilidad y la depuración sobre la descentralización.

9.6 Idempotencia

Todas las operaciones de escritura expuestas por el API Gateway aceptan una cabecera Idempotency-Key (UUID generado por el cliente). El MI persiste la clave en su store y, si la misma clave se presenta dentro de 24 horas, retorna el resultado cacheado en lugar de re-ejecutar. Los microservicios downstream validan idempotencia por aggregate id en su propia base. Los eventos NATS incluyen el correlation_id del flujo original, permitiendo a los suscriptores detectar duplicados.

9.7 Catálogo de eventos del bus NATS

ID evento	Subject NATS	Publicador	Suscriptores
EVT-01	telemedicina.identidad.usuario.creado.v1	WSO2 IS → Identidad	Pacientes, Profesionales
EVT-02	telemedicina.identidad.usuario.bloqueado.v1	WSO2 IS → Identidad	Auditoría, Notificaciones
EVT-03	telemedicina.paciente.creado.v1	Pacientes	Auditoría
EVT-04	telemedicina.paciente.consentimiento.otorgado.v1	Pacientes	Integraciones, Auditoría
EVT-05	telemedicina.profesional.verificado.v1	Profesionales	Agenda, Auditoría
EVT-06	telemedicina.cita.reservada.v1	Agenda	Pagos, Notificaciones, Video
EVT-07	telemedicina.cita.confirmada.v1	Agenda	Notificaciones
EVT-08	telemedicina.cita.cancelada.v1	Agenda	Pagos, Notificaciones
EVT-09	telemedicina.cita.checkin.v1	Agenda	Video, Expediente
EVT-10	telemedicina.cita.noshow.v1	Agenda	Pagos, Notificaciones
EVT-11	telemedicina.video.sesion.iniciada.v1	Video	Auditoría
EVT-12	telemedicina.video.sesion.finalizada.v1	Video	Expediente, Notificaciones
EVT-13	telemedicina.expediente.nota.creada.v1	Expediente	Recetas, Integraciones, Auditoría
EVT-14	telemedicina.expediente.acceso.registrado.v1	Expediente	Auditoría
EVT-15	telemedicina.receta.emitida.v1	Recetas	Notificaciones, Integraciones, Auditoría

ID evento	Subject NATS	Publicador	Suscriptores
EVT-16	telemedicina.pago.autorizado.v1	Pagos	Agenda
EVT-17	telemedicina.pago.capturado.v1	Pagos	Notificaciones, Auditoría
EVT-18	telemedicina.cfdi.generado.v1	Pagos	Notificaciones, Auditoría
EVT-19	telemedicina.notificacion.enviada.v1	Notificaciones	Auditoría
EVT-20	telemedicina.alerta.critica.laboratorio.v1	Integraciones	Notificaciones

Tabla 14. Catálogo de eventos (EVT-01 a EVT-20) en NATS JetStream.

10. Diagramas de Secuencia de Flujos Críticos

Se documentan cuatro diagramas de secuencia de los flujos críticos. Cada diagrama muestra la interacción entre el cliente (BFF), el API Gateway (APIM), el Micro Integrador (MI), WSO2 IS y los microservicios de dominio. Las flechas numeradas indican el orden de invocación.

10.1 SEQ-01: Reserva de cita con autorización diferida

El flujo de reserva de cita combina cinco servicios con autorización de pago diferida (la captura real del pago ocurre al finalizar la videoconsulta, no al reservar).

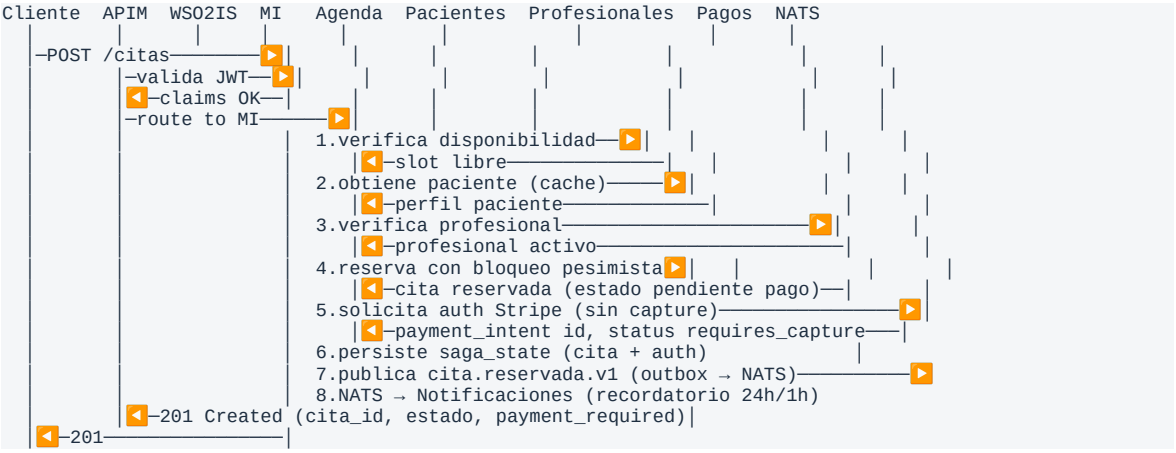


Figura 2. Diagrama de secuencia SEQ-01: reserva de cita con autorización diferida.

10.2 SEQ-04: Emisión de receta electrónica firmada

Este flujo dispara la verificación reforzada del profesional (MFA step-up ya realizado al login), la emisión de la receta en el servicio, la firma electrónica avanzada vía e.firma/SAT, el

envío al paciente y a farmacia asociada opcional, y el reporte a COFEPRIS si la receta incluye controlados.

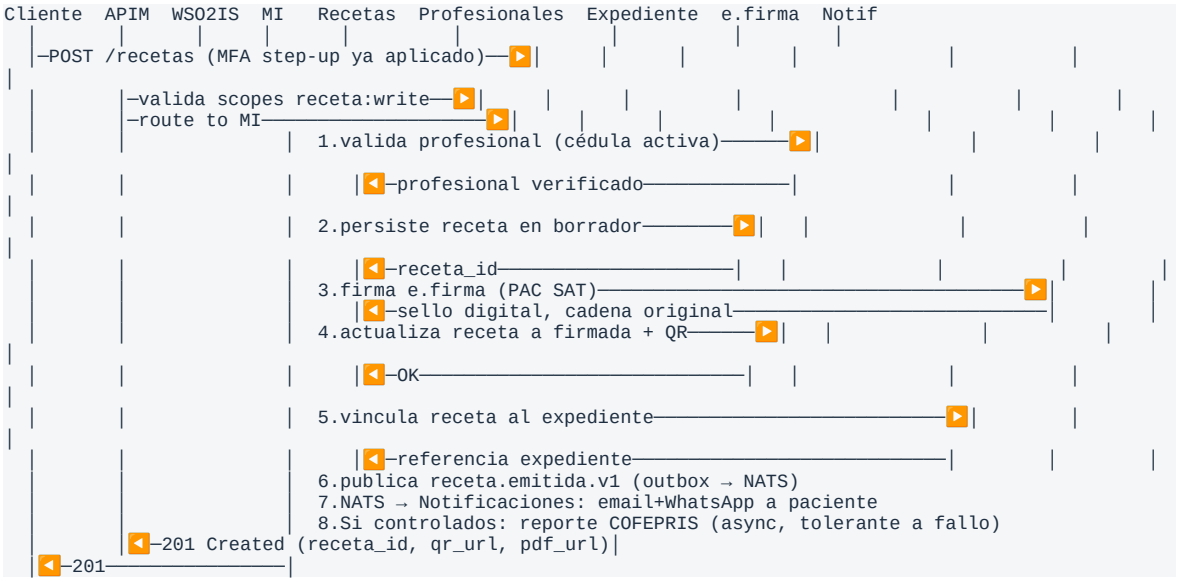


Figura 3. Diagrama de secuencia SEQ-04: emisión de receta electrónica firmada.

10.3 SEQ-05: Captura de pago con CFDI

Este flujo se dispara al cierre exitoso de la videoconsulta. Captura el pago previamente autorizado, genera el CFDI 4.0 vía PAC autorizado por SAT, vincula la factura al expediente como referencia contable, y notifica al paciente con el PDF del CFDI.

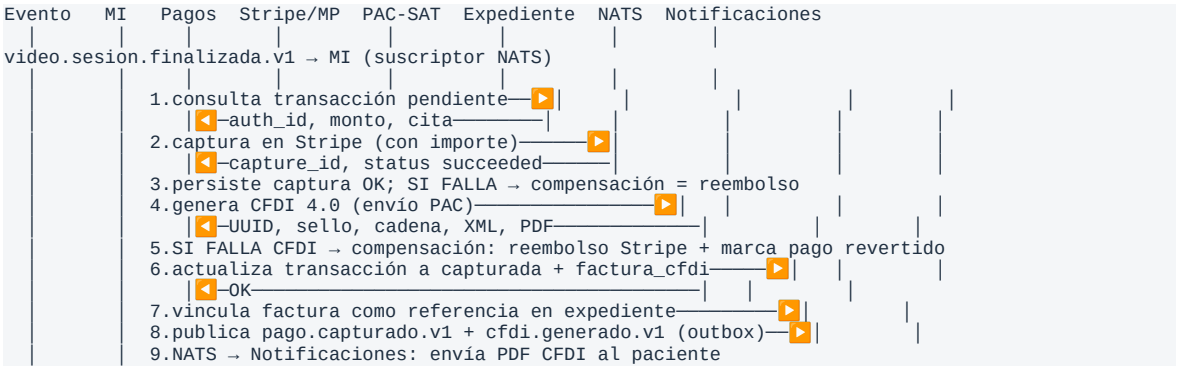


Figura 4. Diagrama de secuencia SEQ-05: captura de pago con CFDI.

10.4 SEQ-07: Exportación FHIR del expediente

Una aplicación SMART on FHIR solicita el bundle del expediente de un paciente (con consentimiento explícito otorgado en WSO2 IS). El Micro Integrador compone el Bundle FHIR agregando recursos de varios servicios internos.



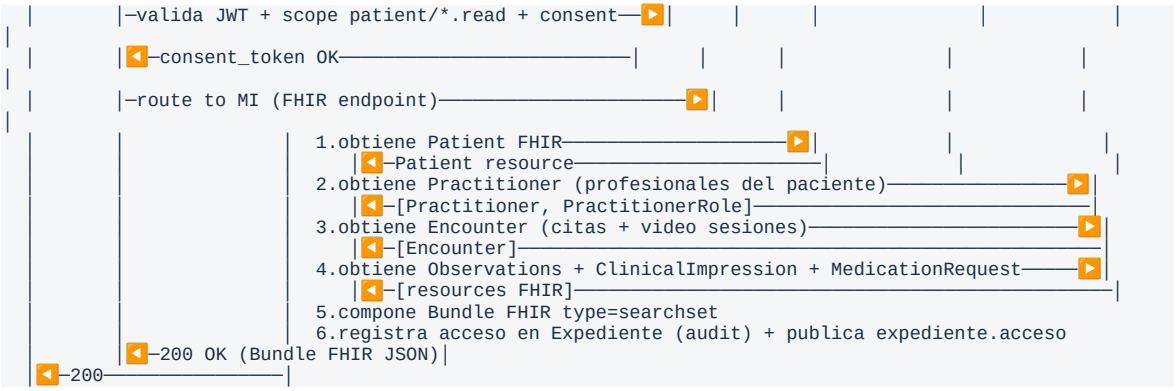


Figura 5. Diagrama de secuencia SEQ-07: exportación FHIR del expediente.

11. Seguridad Aplicada

La seguridad se aplica por capas conforme al ADR-001 y al RNF-01 a RNF-03 del documento de requisitos. Esta sección detalla la implementación concreta de cada capa en el contexto del stack WSO2.

11.1 Defensa en profundidad

Capa perímetro: AWS WAF con reglas OWASP Core Rule Set 4.0, protección DDoS vía AWS Shield Advanced, rate limiting en el ALB. Capa red: VPC con subredes públicas/privadas, NetworkPolicies de Kubernetes por namespace, mTLS entre servicios gestionado por Istio service mesh o Linkerd. Capa aplicación: validación de entradas con class-validator, sanitización de outputs, cabeceras de seguridad en cada respuesta, CSRF tokens para formularios administrativos.

Capa identidad: WSO2 IS con MFA obligatoria para profesionales y administradores, FIDO2 donde aplique, federación SAML/OIDC, step-up authentication para operaciones sensibles, revocación inmediata de tokens. Capa datos: cifrado AES-256 en reposo para RDS, S3 y backups gestionado por KMS; cifrado TLS 1.3 en tránsito; tokenización PCI-DSS para datos de tarjeta (jamás se tocan los PAN); seudonimización de datos en entornos no productivos; anonimización para analytics. Capa observabilidad: SIEM con OpenSearch Dashboards, alertas a PagerDuty, retención de logs 5 años. Capa procesos: gestión de incidentes con runbooks, parcheo automatizado vía renovación de imágenes base mensual, capacitación anual en HIPAA y LFPDPPP para todo el equipo.

11.2 Gestión de secretos

Los secretos se gestionan en HashiCorp Vault desplegado en el clúster o en AWS Secrets Manager como alternativa gestionada. Los pods recuperan secretos al arranque mediante el Vault Agent Sidecar Injector o External Secrets Operator, nunca se montan como variables de texto plano en manifiestos. La rotación de secretos sigue políticas automáticas: claves de bases de datos cada 90 días, claves de conectores externos cada 30 días, certificados TLS gestionados por cert-manager con rotación automática cada 60 días.

11.3 Cifrado y gestión de claves

Las claves maestras residen en AWS KMS o GCP KMS con HSM backing. WSO2 IS gestiona sus claves de firma JWT mediante keystore interno respaldado por KMS, con rotación anual y publicación del JWKS endpoint con cache de 1 hora. Los datos de pacientes en S3 se cifran con SSE-KMS usando CMK por tenant en Fase 4. Las grabaciones de video se cifran con clave única por sesión, derivada y destruida tras el periodo de retención configurado por consentimiento.

11.4 Auditoría de identidad

WSO2 IS registra todos los eventos de identidad (login, MFA, consentimiento, federación, cambio de claims) en su bitácora interna, que se exporta vía Filebeat al pipeline centralizado de logs (OpenSearch). El microservicio de auditoría consume adicionalmente los eventos EVT-02 (usuario bloqueado), EVT-14 (acceso a expediente), EVT-15 (receta emitida) y EVT-17 (pago capturado) para componer una vista unificada de la actividad sensible. La bitácora resultante es append-only con hash encadenado, conforme al RNF-14.

11.5 Cumplimiento HIPAA, RGPD, NOM-024 e ISO 27001

HIPAA: BAAs firmados con AWS/GCP, Stripe (PCI scope), Twilio, Meta (WhatsApp) y PAC-SAT. Todos los subcontratistas con acceso a PHI están bajo BAA. RGPD: DPIA por módulo que procese datos sensibles; derechos ARSO implementados (acceso, rectificación, supresión con reserva legal, oposición); DPO designado en Fase 2. NOM-024: expediente clínico electrónico conforme al estándar de intercambio, exportación FHIR; bitácora inmutable; retención mínima 5 años; consentimiento informado digital firmado. ISO 27001: certificación objetivo en Fase 3; controles del Anexo A mapeados a implementación concreta.

12. Infraestructura y Despliegue

La infraestructura se despliega en AWS (región preferente us-east-1 con dr en us-west-2) o GCP equivalente, sobre Kubernetes gestionado (EKS o GKE). Toda la infraestructura se provisiona con Terraform, los manifiestos Kubernetes con Helm, y los despliegues se sincronizan con ArgoCD en patrón GitOps.

12.1 Topología Kubernetes

El clúster se compone de tres node pools: sistema (componentes de plataforma: WSO2, Istio, cert-manager, External Secrets, Prometheus, Loki), servicios (microservicios de dominio: 12 servicios), y data (Redis cluster, sidecars de OpenSearch, NATS). Los nodos se etiquetan y los pods se afinan con nodeSelector y tolerations para evitar mezclar cargas incompatibles.

12.2 Namespaces y NetworkPolicy

Se definen los siguientes namespaces con NetworkPolicy que aísla el tráfico entre ellos: wso2 (Identity Server, API Manager, Micro Integrator), bff (tres BFF Next.js), core (12 microservicios de dominio), data (Redis, NATS, OpenSearch clients), observability (Prometheus, Grafana, Loki, OpenTelemetry Collector), ingress (NGINX Ingress, cert-manager), ci-cd (ArgoCD, runners). El namespace wso2 acepta tráfico solo desde bff e ingress; core acepta tráfico solo desde wso2; data acepta solo desde core y wso2.

Namespace	Componentes	Tráfico entrante	Replicas típicas
wso2	IS, APIM, MI	bff, ingress	IS 2, APIM-GW 3, MI 3
bff	BFF paciente, profesional, admin	ingress	3 cada uno
core	12 microservicios dominio	wso2	2-6 según carga
data	Redis, NATS, OS clients	core, wso2	Redis 6, NATS 3
observability	Prometheus, Grafana, Loki, OTel	interno	HA
ingress	NGINX, cert-manager	ALB	3
ci-cd	ArgoCD	GitHub webhook	2

Tabla 15. Namespaces de Kubernetes y políticas de tráfico.

12.3 Helm charts y GitOps

Cada componente se empaqueta como Helm chart en repositorio dedicado dentro del monorepo o repos separados según preferencia del equipo. Los values se versionan por entorno (dev, staging, prod) en un repositorio gitops dedicado. ArgoCD sincroniza el estado declarado con el clúster, con sync waves que respetan dependencias (data → wso2 → core → bff). Los despliegues usan estrategia rolling con readiness probes estrictas; los despliegues críticos (WSO2 IS, API Gateway) usan blue-green vía Argo Rollouts.

12.4 Topología multi-región

En Fase 1 y 2 se opera single-region con DR warm standby en la región secundaria: RDS cross-region read replica, S3 cross-region replication, EKS standby con escalado a cero. En Fase 3 se promueve a activo-pasivo con failover automatizado vía Route53 health checks. En Fase 4 se evalúa activo-activo para los componentes stateless (BFF, microservicios, WSO2 APIM Gateway, WSO2 MI) con los stores de datos manteniendo activo-pasivo por complejidad de consistencia.

12.5 Backup y disaster recovery

Backups RDS automatizados cada 15 minutos (retención 30 días), snapshot diario retenido 90 días para datos clínicos. Backups Redis via BGSAVE cada hora replicados a S3 cross-region. OpenSearch snapshots diarios a S3. Backups de configuración WSO2 IS (keystores, configs) diarios cifrados a S3. Ejercicios de DR trimestrales con failover real a la región secundaria y verificación de RTO 4h / RPO 15min conforme a RNF-05 y RNF-06.

13. Observabilidad

La observabilidad se implementa sobre los tres pilares: logs estructurados, métricas y trazas distribuidas, con correlación mediante correlation_id y traceparent propagados por WSO2 APIM, WSO2 MI y los microservicios.

13.1 Logs estructurados

Todos los componentes emiten logs en JSON estructurado con campos estándar: timestamp, level, service, correlation_id, trace_id, span_id, message, y campos contextuales (user_id enmascarado, tenant_id, cita_id cuando aplique). Los logs se recogen con Filebeat/FluentBit y se envían a Loki para logs generales y a OpenSearch para logs de

auditoría y de WSO2. La retención es de 30 días en caliente, 90 días en tibio, 5 años para logs de auditoría conforme a RNF-14.

13.2 Métricas

Cada microservicio expone /metrics en formato Prometheus con métricas RED (Rate, Errors, Duration) por endpoint y métricas de negocio (citas reservadas por minuto, videoconsultas activas, pagos capturados). WSO2 APIM expone métricas de gateway (requests por API, latencia, throttled). WSO2 MI expone métricas de orquestación (secuencias por minuto, fallos, duración). WSO2 IS expone métricas de identidad (logins, MFA success rate, federated logins). Prometheus scrapea todos los targets; Grafana visualiza con dashboards por servicio y un dashboard ejecutivo con KPIs operativos.

13.3 Trazas distribuidas

OpenTelemetry instrumenta WSO2 APIM, WSO2 MI, WSO2 IS y todos los microservicios NestJS vía SDK. El traceparent se propaga en cabeceras W3C Trace Context. Las trazas se envían a un backend Jaeger o AWS X-Ray. Cada flujo crítico (SEQ-01 a SEQ-08) tiene su service map y SLI/SLO definidos: latencia p95 objetivo por secuencia, tasa de error, y budget de error mensual conforme a SRE.

13.4 Dashboards y alertas

Dashboards operativos: SLO por servicio, top errores, latencia por endpoint, eventos NATS lag, hilos de BullMQ. Dashboards de seguridad: intentos de login fallidos por IP, MFA failures, accesos a expedientes sensibles, throttled requests. Dashboards de negocio: consultas activas, no-shows, pagos capturados, NPS en tiempo real. Las alertas se enrutan a PagerDuty con escalado on-call 24/7 para servicios críticos (WSO2 IS, APIM, MI, Expediente, Pagos) y horario laboral para los demás.

Dashboard	Audiencia	Refresh	Origen datos
SLO por servicio	SRE, Tech Lead	1 min	Prometheus
Top errores 24h	SRE, dev on-call	1 min	Loki + Prometheus
Auditoría de identidad	Security, Compliance	5 min	OpenSearch
Consultas activas	Producto, Ops	1 min	Prometheus + API
Costos cloud	FinOps	1 día	AWS Cost Explorer
SAGAS en curso	SRE	30 s	WSO2 MI metrics

Tabla 16. Dashboards operativos y de negocio.

14. Plan de Despliegue por Fases

El despliegue del diseño se mapea al roadmap en cinco fases definido en el documento de requisitos. Cada fase agrega componentes sobre la base de la anterior, respetando dependencias técnicas y regulatorias.

14.1 Fase 0 - Descubrimiento y Diseño (4 semanas)

Esta fase produce el presente TDD en su versión final aprobada, el DPIA, la selección del cloud provider y la firma del BAA. Adicionalmente se aprovisiona el clúster Kubernetes base con namespaces, el repositorio GitOps, el pipeline CI/CD y el entorno de WSO2 IS en su versión mínima para pruebas de concepto de federación SAML y OIDC con un IdP institucional piloto.

14.2 Fase 1 - MVP (12 semanas)

Se despliegan WSO2 IS en HA, WSO2 APIM Gateway, WSO2 MI en HA. Microservicios: identidad, pacientes, profesionales, agenda, video, expediente, notificaciones, auditoría. BFF paciente y profesional. Sin pagos ni recetas ni mensajería ni integraciones FHIR externas. Objetivo: videoconsulta + agenda + expediente básico funcionales con 100 usuarios simultáneos.

14.3 Fase 2 - Beta Cerrada (8 semanas)

Se incorporan los microservicios restantes: pagos (con Stripe/MP y CFDI), recetas (con e.firma), mensajería, integraciones (FHIR interno y SMART on FHIR piloto con una app externa). WSO2 MI despliega las secuencias SEQ-04, SEQ-05, SEQ-06, SEQ-07, SEQ-08. BFF admin para gestión institucional. 50 usuarios beta reales.

14.4 Fase 3 - Piloto Productivo (8 semanas)

Despliegue multi-región activo-pasivo con failover automatizado. Certificación NOM-024 obtenida o en proceso avanzado. Auditoría externa de cumplimiento HIPAA e ISO 27001. Primera institución productiva operando con SSO federado SAML u OIDC hacia su IdP. SLA 99,95% medido durante 4 semanas consecutivas.

14.5 Fase 4 - Escalamiento (12 semanas)

Habilitación multi-tenant completa con RLS por tenant y CMK por tenant en S3. White-label funcional (Dev Portal y BFF branded por institución). Marketplace APIs público con tiers bronze/silver/gold/platinum. SDKs publicados. Primera expansión a otra región de México. Activación de servicio de streaming de eventos FHIR vía webhooks para partners.

15. Riesgos Técnicos y Mitigaciones

Se identifican doce riesgos técnicos específicos del presente diseño. Cada riesgo se evalúa con probabilidad (Alta/Media/Baja) e impacto (Crítico/Alto/Medio/Bajo) y se asigna mitigación concreta.

ID	Riesgo	Prob.	Impacto	Mitigación
RT-01	Dependencia de WSO2 como single vendor para tres capas	Media	Alto	Contrato con soporte enterprise; designar un WSO2 admin; mantener knowledge base interna
RT-02	Curva de aprendizaje del equipo en WSO2 MI y secuencia de mediación	Alta	Medio	Capacitación inicial 2 semanas; pairing con consultor externo Fase 1; documentar patrones
RT-03	Micro Integrator como single point of failure en sagas críticas	Media	Crítico	Despliegue HA 3+ réplicas; saga_state persistido en MySQL compartido; saga recovery automático tras restart
RT-04	Latencia agregada por capa APIM+MI en flujos críticos	Media	Alto	Medir latencia por secuencia; budgets SLO p95 < 300ms; optimizar secuencias más largas con cache y paralelización
RT-05	Fuga cross-tenant si RLS PostgreSQL se omite en alguna tabla	Baja	Crítico	Tests automatizados de aislamiento por tenant en cada PR; linter que valide presencia de tenant_id en migraciones
RT-06	Inconsistencia dual-write entre DB y NATS si patrón outbox falla	Baja	Alto	Worker outbox con idempotencia; alerta si lag > 60s; reconciliation job horario
RT-07	Caída del PAC-SAT durante generación de CFDI deja pago capturado sin factura	Media	Alto	Cola de reintentos con backoff; compensación manual o nota de crédito; alerta inmediata

ID	Riesgo	Prob.	Impacto	Mitigación
RT-08	e.firma del médico revocada durante validez de recetas emitidas	Baja	Alto	Verificación periódica de estatus de certificados; invalidación de recetas afectadas con notificación
RT-09	Webhook de WSO2 IS (SCIM) perdido deja perfil interno sin sincronizar	Media	Medio	Reconciliation batch diario que compare user_ids WSO2 vs perfiles internos
RT-10	Costo cloud superior por uso intensivo de OpenSearch para auditoría	Media	Medio	Index templates optimizados; lifecycle management ILM; rotación a S3 cold storage tras 90 días
RT-11	Versiones de WSO2 descontinuadas durante la vida del proyecto	Baja	Alto	Política de LTS; upgrade planificado cada 18 meses; mantener compatibilidad con configs versionadas
RT-12	Migración multi-región activo-activa introduce consistencia eventual problemática	Media	Crítico	En Fase 4 mantener stores activo-pasivo; solo stateless en activo-activo; contraste con DPIA

Tabla 17. Catálogo de riesgos técnicos (RT-01 a RT-12) con plan de mitigación.

Anexo A. Catálogo de APIs Públicas

El siguiente catálogo consolida los endpoints públicos expuestos por WSO2 API Manager, con su método HTTP, backend, scopes requeridos y la secuencia de mediación del Micro Integrator que lo sirve cuando aplica.

Endpoint público	Método	Backend	Scopes	Secuencia MI
/pacientes/me	GET	Pacientes	perfil:read	—
/citas	POST	Agenda	cita:write	SEQ-01
/citas/{id}	GET	Agenda	cita:read	Aggregator MI
/citas/{id}/cancelar	POST	Agenda	cita:write	—
/video/sesiones/{id}	POST	Video	video:session	—
/video/sesiones/{id}/finalizar	POST	Video	video:admin	SEQ-03
/recetas	POST	Recetas	receta:write	SEQ-04
/recetas/{id}/qr	GET	Recetas	receta:read	—
/pagos/{id}/capturar	POST	Pagos	pago:write	SEQ-05

Endpoint público	Método	Backend	Scopes	Secuencia MI
/pagos/{id}/reembolsar	POST	Pagos	pago:write	SEQ-06
/mensajes	POST	Mensajería	mensaje:write	—
/interconsultas	POST	Mensajería	mensaje:write	SEQ-08
/fhir/Patient/{id}	GET	Integraciones	patient/Patient.read	—
/fhir/Patient/{id}/$everything	GET	Integraciones	patient/*.read	SEQ-07
/fhir/MedicationRequest	POST	Integraciones	patient/MedicationRequest.write	—
/admin/usuarios	GET	Administración	admin:tenant:read	—
/admin/usuarios	POST	Administración	admin:tenant:write	—

Tabla A1. Catálogo de APIs públicas del API Gateway.

Anexo B. Glosario y Referencias

B.1 Glosario de términos técnicos

Se presenta el glosario de términos, siglas y acrónimos específicos del presente documento de diseño técnico, complementario del glosario general del documento de análisis y requisitos v1.0.

Término	Definición
ADR	Architecture Decision Record, registro formal de una decisión arquitectónica significativa
BFF	Backend for Frontend, backend dedicado por tipo de cliente
Claim	Atributo del usuario incluido en un token JWT
CloudEvents	Especificación estandarizada para describir eventos en formato común
CMK	Customer Managed Key, clave de cifrado gestionada por el cliente en KMS
FIDO2/WebAuthn	Estándar de autenticación con factores hardware o platform authenticators
IdP	Identity Provider, proveedor de identidad
JWKS	JSON Web Key Set, endpoint que publica las claves públicas para validar JWT
mTLS	Mutual TLS, autenticación bidireccional con certificados cliente y servidor

Término	Definición
OIDC	OpenID Connect, protocolo de identidad sobre OAuth 2.0
Outbox pattern	Patrón de publicación transaccional de eventos usando tabla local
PAC	Proveedor Autorizado de Certificación (SAT, México) para CFDI
RLS	Row-Level Security de PostgreSQL, políticas de acceso por fila
Saga	Patrón de transacción distribuida con compensaciones
SCIM	System for Cross-domain Identity Management, protocolo de provisioning
SMART on FHIR	Iniciativa para apps clínicas basadas en FHIR con autorización OAuth
Step-up authentication	Solicitud de factor adicional en runtime según sensibilidad
TOTP	Time-based One-Time Password, código de un solo uso basado en tiempo
WSO2 IS	WSO2 Identity Server, producto IdP de WSO2
WSO2 APIM	WSO2 API Manager, producto gateway + portal de WSO2
WSO2 MI	WSO2 Micro Integrator, producto de orquestación de WSO2

Tabla B1. Glosario de términos técnicos del TDD.

B.2 Referencias técnicas y normativas

Las siguientes referencias técnicas y normativas sustentan el presente documento de diseño técnico:

1. Documento de Análisis y Requisitos v1.0. Plataforma de Telemedicina. Repositorio interno: docs/01-analisis-requisitos/.

2. WSO2 Identity Server 7.x Documentation. WSO2 Inc. URL: https://is.docs.wso2.com/

3. WSO2 API Manager 4.x Documentation. WSO2 Inc. URL: https://apim.docs.wso2.com/

4. WSO2 Micro Integrator 4.x Documentation. WSO2 Inc. URL: https://mi.docs.wso2.com/

5. OpenID Connect Core 1.0 specification. OpenID Foundation. URL: https://openid.net/specs/openid-connect-core-1_0.html

6. SCIM 2.0 Protocol Specification. IETF RFC 7644. URL: https://datatracker.ietf.org/doc/html/rfc7644

7. CloudEvents 1.0 Specification. CNCF. URL: https://github.com/cloudevents/spec

8. NATS JetStream Documentation. Synadia. URL: https://docs.nats.io/nats-concepts/jetstream

9. FHIR R4 Specification. HL7 International. URL: https://hl7.org/fhir/R4/

10. SMART App Launch Framework. HL7 / Boston Children's Hospital. URL: https://build.fhir.org/ig/HL7/smart-app-launch/

11. W3C WebAuthn Recommendation. W3C. URL: https://www.w3.org/TR/webauthn/

12. NIST SP 800-63B Digital Identity Guidelines. NIST. URL: https://pages.nist.gov/800-63-3/sp800-63b.html

13. PostgreSQL Row Level Security Documentation. PostgreSQL. URL: https://www.postgresql.org/docs/current/ddl-rowsecurity.html

14. OpenTelemetry Specification. CNCF. URL: https://opentelemetry.io/docs/specs/

15. C4 Model for Software Architecture. Simon Brown. URL: https://c4model.com/